Agent Design Report

A Governed Multi-Agent Architecture for Autonomous E-Commerce and Accounting Operations

Design specification grounded in the inspected project repository

Document type:	Agent design and implementation-status report
Architecture:	Seven governed agents (Orchestrator, E-Commerce, Inventory, Accounting, Risk/Anomaly, Governance, Audit)
Evidence base:	`reports/dataset_report/` (dataset analysis package), `reports/dataset_selection_table/` (project/dataset overview), `datasets/` (Olist CSV, Online Retail II XLSX)
Diagrams:	`figures/` (generated by `figures/make_figures.py`)
Date:	October 16, 2026

Integrity note. This report distinguishes, at every point, (i) components that are implemented *in the repository, (ii) components that are* designed but not implemented, *and (iii) components that are* proposed for future work. *Repository inspection found no agent implementation; consequently all seven agents are documented as proposed. This is stated explicitly rather than implied.*

Abstract

This report specifies a seven-agent, governance-first architecture for the research project *Governed Multi-Agent AI for Autonomous E-Commerce and Accounting Operations*, and documents its implementation status strictly from evidence found in the project repository. The seven agents are an Orchestrator/Coordination agent, an E-Commerce Operations agent, an Inventory Management agent, an Accounting agent, a Risk/Anomaly-Detection agent, a Governance/Policy-Enforcement agent, and an Audit/Traceability agent. Repository inspection established that **no agent code, message bus, policy engine, or audit ledger is present**: the only implemented software is (i) a reproducible dataset-analysis and validation package under `reports/dataset_report/analysis/` — which corresponds to the project's prospective *ingestion and validation* layer and which produced the measured facts reused in this report — and (ii) a standard-library dataset inspector under `reports/dataset_selection_table/scripts/`. All seven agents are therefore *proposed*: they have a specified purpose, inputs, outputs, internal workflow, dependencies, governance requirements, failure handling and evaluation plan, but no executable realisation. The architecture operates over two independent, read-only datasets (the Olist Brazilian E-Commerce dataset and Online Retail II), ingested through separate source adapters that must never be record-level merged. Governance is modelled as an explicit lifecycle (request → planning → authorization → execution → validation → risk assessment → human approval → audit → response), and the report states which controls would be enforced in code and which currently remain design recommendations. Evaluation is presented as a *proposed* protocol because no agent experiments have yet been run. Every numerical dataset fact cited here is measured by the accompanying analysis package and reported in the project's *Dataset Report*; no implementation, test result, performance figure, or capability is fabricated.

Contents

Glossary of Abbreviations **3**

1 Executive Summary **3**

2 Introduction and Research Motivation **4**
2.1 Motivation . . . 4
2.2 Why a single unrestricted agent is insufficient . . . 4
2.3 Scope of this report . . . 4

3 System Objectives and Scope **4**
3.1 Objectives . . . 4
3.2 Expected users . . . 5
3.3 Operational scope and assumptions . . . 5
3.4 Prototype versus production . . . 5

4 Overall System Architecture **5**
4.1 Overview . . . 5
4.2 Implementation status of the architecture . . . 6
4.3 Interaction summary . . . 7

5 Individual Agent Specifications **7**
5.1 Agent 1 — Orchestrator / Coordination Agent . . . 7
5.2 Agent 2 — E-Commerce Operations Agent . . . 8
5.3 Agent 3 — Inventory Management Agent . . . 9
5.4 Agent 4 — Accounting Agent . . . 9
5.5 Agent 5 — Risk and Anomaly Detection Agent . . . 10
5.6 Agent 6 — Governance and Policy Enforcement Agent . . . 11
5.7 Agent 7 — Audit and Traceability Agent . . . 12

5.8 Agent comparison 12

6 Inter-Agent Communication and Coordination 13

7 Governance and Security Model 14
7.1 Governance lifecycle 14
7.2 Candidate policy rules 14
7.3 Security principles 14
7.4 What is enforced versus recommended 15

8 Dataset and Agent Mapping 15

9 End-to-End Workflow Scenarios 16
9.1 Scenario A — Order and payment reconciliation 16
9.2 Scenario B — Transaction anomaly investigation 17
9.3 Scenario C — Inventory risk analysis 18
9.4 Scenario D — Governance rejection 19

10 Data Contracts and Audit Schema 20

11 Evaluation Methodology 21

12 Failure Modes and Limitations 22

13 Implementation Roadmap 23

14 Conclusion and Future Work 23

A Repository Inventory and Reproduction 24

Glossary of Abbreviations

Term	Meaning
Agent	A software component with an explicit role, inputs, outputs and a governance-wrapped interface.
Audit event	A prototype-generated, append-only record of an agent action and its decision context.
BRL	Brazilian Real, the currency implied by the Olist marketplace context.
Canonical event	A minimal, source-tagged internal representation shared across adapters where a defensible mapping exists.
Evidence reference	A pointer to the source record(s) (table + key) that justify a decision.
FK	Foreign key.
GBP (£)	Pound sterling, the currency of Online Retail II.
Governance	The set of explicit policies and checks that constrain agent actions (allow / block / escalate).
Human-in-the-loop	A required human approval step for high-impact actions defined by policy.
Orchestrator	The agent that plans tasks, delegates to specialists and consolidates results.
PK	Primary key.
Proposed	Designed in this report but not implemented in the repository.
Implemented	Present as executable code in the repository (evidenced by file and function references).
Source adapter	Read-only component that ingests one dataset and emits canonical events.

1 Executive Summary

The project investigates how a set of specialised, governed software agents can cooperate to perform e-commerce and accounting-related workflows while remaining traceable and auditable. This report turns the project's conceptual agent roles into a concrete design specification, and — critically — separates that specification from the current state of the repository.

What was found. A full inspection of the repository (see Appendix A) identified exactly three substantive content areas: the raw datasets under `datasets/`; a dataset-analysis report and package under `reports/dataset_report/`; and a dataset-selection report and inspector under `reports/dataset_selection_table/`. There is **no agent code** anywhere in the repository: no orchestrator, no message schemas, no policy engine, no audit store, and no test that exercises agent behaviour. The directory `src/` is empty and the root `README.md` is empty. Accordingly, **every one of the seven agents is classified as *proposed*** in this report.

What is implemented. The only executable software relevant to the project is the dataset-analysis package (`reports/dataset_report/analysis/`), which reads the raw files read-only and computes schema validation, candidate-key checks, referential-integrity checks, join-inflation measurement, missingness and duplicate profiling, temporal and monetary summaries, a payment-versus-item reconciliation proxy, business-signal detection, and figures. This package is, in effect, a working prototype of the project's *ingestion and validation* layer and is the foundation on which the agents would be built. A second, independent inspector (`reports/dataset_selection_table/scripts/inspect_datasets.py`) performs a standard-library inventory and quality scan. The analysis package is covered by thirteen unit tests, all of which were executed and pass; these tests, however, validate data metrics, not agent behaviour.

Datasets. Two independent datasets supply evidence: Olist (nine relational CSVs; 1,550,922 rows) as the primary source for multi-table e-commerce workflows, and Online Retail II (two worksheets; 1,067,371 invoice lines) as a complementary transaction-level source. Neither contains inventory stock levels, a general ledger, explicit governance policies, or verified fraud labels. These limitations are inherited by the agent design and are stated throughout.

Governance and audit. Governance is specified as an explicit, versioned lifecycle applied to every agent action, with a separation between *recommendation* and *execution*, and with human approval required for policy-defined high-impact actions. Because none of this is implemented, all such controls are presented as design recommendations, and the report is explicit about the resulting gap between design and enforcement.

2 Introduction and Research Motivation

2.1 Motivation

Business operations such as order fulfilment and accounting reconciliation are inherently *multi-step* and *cross-functional*: a purchase triggers payment capture, fulfilment, invoicing and, later, reconciliation and reporting. Each step has different data, different failure modes and different control requirements. This structure motivates a division of labour among specialised agents [11, 8], each an expert over a narrow slice of the workflow.

2.2 Why a single unrestricted agent is insufficient

A single agent with broad permissions faces three problems that motivate the governance-first design:

1. **Separation of duties.** Accounting and audit controls rely on the principle that the same actor should not both propose and approve a consequential action [3, 10]. A monolithic agent collapses these roles.
2. **Focused validation.** Different tasks require different validations (schema and key integrity for ingestion; amount reconciliation for accounting; distributional and temporal checks for anomaly detection). Specialised agents allow each validation to be explicit and testable.
3. **Traceability.** When one component does everything, it is hard to isolate which step produced a given outcome. An orchestrated set of agents with an audit trail yields finer-grained traceability, which is a prerequisite for trustworthy automation [9].

2.3 Scope of this report

The report specifies the seven-agent architecture, its interfaces, its governance model, its mapping onto the two datasets, four end-to-end scenarios, proposed data contracts, a proposed evaluation protocol, failure modes, and an implementation roadmap. Throughout, it preserves the terminology and dataset findings already established in the project's *Dataset Report* (Deliverable 2, the dataset analysis) and *project overview* (Deliverable 1, the dataset-selection report), and it does not re-derive the dataset analysis.

3 System Objectives and Scope

3.1 Objectives

- **O1.** Provide a coordination layer that decomposes a user request into tasks and routes them to specialised agents.

- **O2.** Ground each specialised agent in the two available datasets through read-only adapters, without fabricating data that the sources do not contain.
- **O3.** Enforce explicit, versioned governance policies on every agent action, with human approval for high-impact actions.
- **O4.** Produce an append-only audit trail that links each recommendation to the evidence that justified it.
- **O5.** Support a reproducible evaluation of coordination, validation, reconciliation and governance behaviour.

3.2 Expected users

The primary users are the research team and a supervisor evaluating the prototype; secondary (future) users are operations and finance staff in a hypothetical adopter. No end-user interface exists at present.

3.3 Operational scope and assumptions

- The system operates in **offline replay mode** over historical datasets; it does not connect to live commerce or accounting systems.
- The two datasets are treated as **independent evidence sources**. They differ in currency (BRL vs. £), period, schema and semantics, and **must not be record-level merged**.
- Monetary and temporal analyses are **analytical**: they are not ledger postings or audited statements.
- Anomalies are **candidate** signals, not fraud determinations.

3.4 Prototype versus production

This is a **research prototype**. It is not a production-ready autonomous business platform: it lacks persistent operations, authentication, real tools, transactional settlement, and any operational deployment. The distinction is maintained deliberately, so that no reader infers production capability from the design.

4 Overall System Architecture

4.1 Overview

The architecture comprises seven agents organised in layers: a coordination layer (Orchestrator); four operational/analytical layers (E-Commerce, Inventory, Accounting, Risk); a governance layer that gates every action; and an audit layer that records every action. Data reach the agents only through read-only source adapters. Figure 1 shows the proposed structure, with **solid borders for the one implemented component** (the ingestion and validation layer) and **dashed borders for all proposed components**.

Figure 1 - Proposed governed multi-agent architecture (dashed = proposed, solid = implemented)

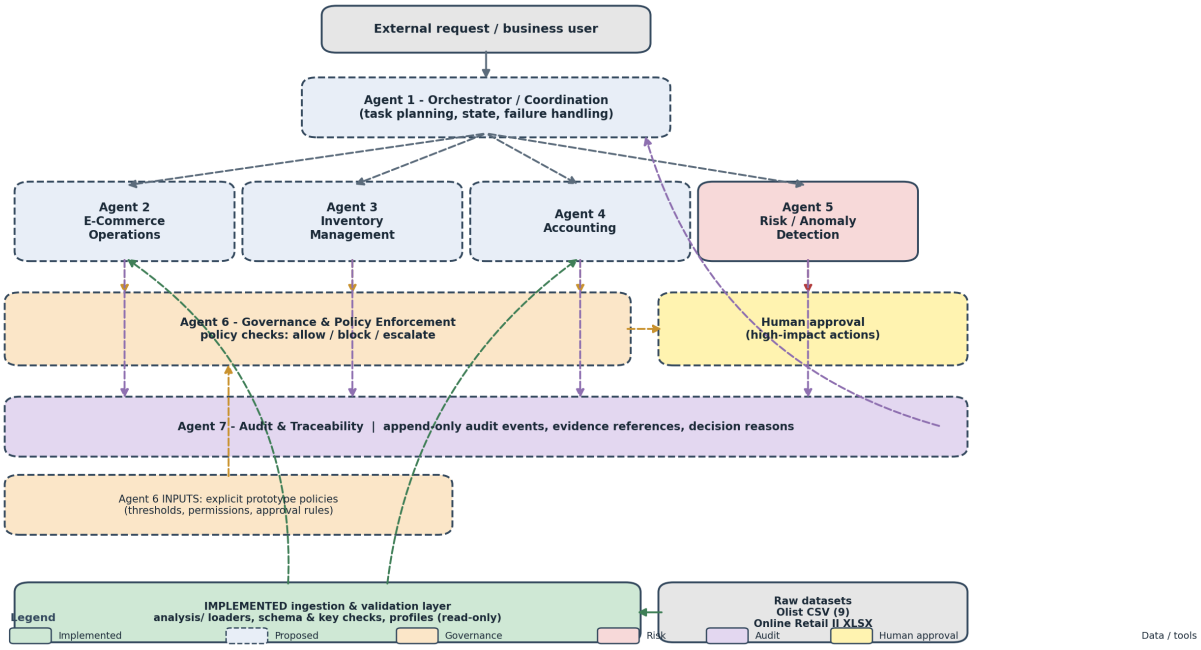


Figure 1: Proposed governed multi-agent architecture. Dashed components are proposed and not implemented; the solid component is the implemented dataset ingestion and validation layer.

4.2 Implementation status of the architecture

Figure 2 summarises the status observed during repository inspection. It separates the seven proposed agents from the two implemented data-layer components. No arrow, message, policy rule or audit record in the architecture is currently realised in code.

Figure 2 - Implementation status observed in the repository

Component	Status (from repository inspection)
Agent 1 Orchestrator	Proposed - not implemented
Agent 2 E-Commerce	Proposed - not implemented
Agent 3 Inventory	Proposed - not implemented
Agent 4 Accounting	Proposed - not implemented
Agent 5 Risk / Anomaly	Proposed - not implemented
Agent 6 Governance	Proposed - not implemented
Agent 7 Audit	Proposed - not implemented
Ingestion & validation layer	Implemented (data layer)
Dataset inspector (Deliverable 1)	Implemented (data layer)

Figure 2: Implementation status per architectural component, observed directly from the repository. The data layer is implemented; all agents are proposed.

4.3 Interaction summary

At a high level, a request enters the Orchestrator (Agent 1). The Orchestrator plans tasks and invokes the relevant operational agents (Agents 2–5). Each operational agent reads canonical events derived by the source adapters, produces a structured result, and submits any consequential action to the Governance agent (Agent 6). The Governance agent returns allow/block/escalate, optionally requiring human approval. Every action and decision is handed to the Audit agent (Agent 7). The Orchestrator then consolidates a recommendation for the user. The governance and audit interactions are described in Sections 6 and 7.

5 Individual Agent Specifications

This section specifies each agent using a common structure: purpose; inputs; outputs; internal workflow; tools and data sources; interactions; governance requirements; error handling; implementation status; and limitations. The implementation status is repeated verbatim from inspection: **all seven agents are proposed / not implemented**. Where an agent could reuse the implemented analysis package, this is noted explicitly.

5.1 Agent 1 — Orchestrator / Coordination Agent

Purpose and responsibilities. Receive a user request, translate it into structured tasks, determine which specialised agents are required, sequence their execution, maintain workflow state, detect failures/timeouts/conflicts, ensure governance checks are invoked, and return a consolidated, traceable result.

Inputs (schema). `UserRequest` (see Section 10): a natural-language or structured goal, requester identity, and an optional scope (dataset, entity, time window).

Outputs (schema). `WorkflowResult`: the consolidated recommendation, the ordered task list with per-task status, the set of agent results, the governance decisions, and the audit event identifiers that reference the supporting evidence.

Internal workflow. (i) Parse and validate the request; (ii) decompose into a directed acyclic task graph; (iii) resolve data scope through the source adapters; (iv) invoke agents in dependency order; (v) route consequential actions through governance; (vi) on success, consolidate; on failure, apply the fallback policy (retry, degrade, or escalate); (vii) record the workflow to the audit trail.

Tools, APIs, libraries, data sources. No orchestration framework is present. The Orchestrator would be implemented over the implemented analysis layer for data access and a to-be-built task/message runtime. Candidate libraries would be chosen by the implementer; none is currently in the repository.

Interactions. Calls Agents 2–5; consults Agent 6 before any consequential action; writes to Agent 7; returns results to the user.

Governance and validation. Must not execute consequential actions directly; must route them through Agent 6; must attach evidence references to every recommendation.

Error handling and fallback. Detects agent failure, timeout, and contradictory outputs; degrades gracefully by returning partial results clearly marked as incomplete; escalates to a human when policy requires.

Implementation status. Proposed. No orchestration code exists. Related implemented capability: none (data access only, via the analysis package).

Limitations and future improvements. Coordination behaviour cannot be validated from historical data; it must be exercised by the prototype and evaluated through its own traces. Circular delegation must be prevented by construction (a task graph with a depth limit).

5.2 Agent 2 — E-Commerce Operations Agent

Purpose and responsibilities. Analyse orders, customers, products, sellers, payments and reviews; support order-related workflows; identify incomplete or inconsistent order information; provide structured operational recommendations; coordinate with the Inventory, Accounting, Risk and Governance agents.

Inputs (schema). Canonical order/transaction events and line items derived from Olist (orders joined to items, payments, reviews) and from Online Retail II (invoice lines). Relevant Olist fields include `order_id`, `order_status`, the order timestamps, `payment_type`, `payment_value` and `review_score`; Online Retail II fields include `Invoice`, `StockCode`, `Quantity`, `InvoiceDate` and `Country`.

Outputs (schema). `OperationsSummary` and `OperationsRecommendation`: order/transaction volumes, status distributions, sales aggregates, cancellation analysis, and flagged completeness/inconsistency issues, each with evidence references.

Internal workflow. Retrieve scoped records; compute aggregates (status distribution, monthly volume, category revenue, cancellation rate); apply completeness checks (e.g. orders lacking items/payment/review); emit structured findings.

Tools, APIs, libraries, data sources. The implemented analysis package already computes the underlying aggregates: Olist status/payment/review value counts, monthly order volumes and category revenue (`analysis/olist.py`; `analysis/figures.py`), and Online Retail II monthly activity and cancellation metrics (`analysis/online_retail.py`). No live e-commerce access exists.

Interactions. Receives tasks from Agent 1; shares demand signals with Agent 3; shares transaction amounts with Agent 4; shares completeness/inconsistency signals with Agent 5; submits recommendations to Agent 6; logs to Agent 7.

Governance and validation. Read-only analysis may proceed under authorization; any recommendation that would change a record must go through Agent 6.

Error handling and fallback. Returns explicit "not available in source dataset" markers rather than imputing missing fields; aborts a task if a required schema is missing (mirroring the implemented schema checks).

Implementation status. Proposed as an agent. The analytical computations it would expose are *partially implemented* as library functions in the analysis package, but there is no agent wrapper, interface, or workflow.

Limitations and future improvements. Olist has no marketing/session/acquisition data; Online Retail II has no order-status concept and 22.77% of lines lack a customer identifier, limiting customer-centric analysis.

5.3 Agent 3 — Inventory Management Agent

Purpose and responsibilities. Analyse product and order-item data to characterise demand; identify possible stock-related risks and availability concerns; produce demand indicators and inventory recommendations *where the data supports them*; communicate findings to the Orchestrator and E-Commerce agent.

Inputs (schema). Olist order items (`product_id`, `order_item_id`, `price`, `freight_value`, `shipping_limit_date`) and product attributes; Online Retail II lines (`StockCode`, `Quantity`, `InvoiceDate`). No stock-on-hand field exists in either source.

Outputs (schema). `DemandIndicator` per product (units sold, demand variability) and a *qualified* `InventoryRecommendation` that explicitly states it is a demand proxy, not a replenishment decision.

Internal workflow. Aggregate units sold per product; compute variability and a days-without-sales proxy; rank products; attach an explicit "no stock data" limitation to every recommendation.

Tools, APIs, libraries, data sources. The implemented package computes per-product category revenue and value counts (`analysis/olist.py`); quantity/price distributions and monthly activity for Online Retail II (`analysis/online_retail.py`).

Interactions. Consumes canonical lines; receives demand context from Agent 2; flags data gaps to Agent 6; logs to Agent 7.

Governance and validation. Must not present demand proxies as actual stock positions; any downstream action requires policy validation.

Error handling and fallback. If product attributes are missing (610 products lack category metadata in Olist; 1.85%), it degrades to line-level quantity analysis and marks the gap.

Implementation status. Proposed. Only the underlying demand aggregations exist; no inventory logic, stock model, or replenishment reasoning is implemented.

Limitations and future improvements. Neither dataset contains stock quantities, warehouse records, replenishment history, or inventory snapshots. Inventory management is therefore an *analytical prototype only*; real stock decisions would require an external inventory source. Listed as a hard limitation throughout.

5.4 Agent 4 — Accounting Agent

Purpose and responsibilities. Analyse payment records, order values and transaction totals; reconcile related records where the fields permit; identify discrepancies requiring investigation; produce structured accounting summaries; send suspicious discrepancies to the Risk agent and relevant decisions to the Governance agent.

Inputs (schema). Olist `order_payments` (`payment_value`, `payment_type`, `payment_installments`) and `order_items` (`price`, `freight_value`); Online Retail II line value (`Quantity` × `Price`), cancellations and adjustments.

Outputs (schema). `ReconciliationResult` (order identifier, item total, payment total, absolute difference, match flag) and `AccountingSummary` (aggregate totals clearly labelled as analytical, not ledger postings).

Internal workflow. Sum item value (price + freight) per order; sum payment value per order; join on `order_id`; compute differences; flag mismatches beyond a threshold; forward flagged cases.

Tools, APIs, libraries, data sources. The implemented package already computes an order-level reconciliation proxy: of the 98,665 orders with both items and payments, 98,287 (99.62%) agree within one cent, mean absolute difference 0.0332 BRL (`analysis/olist.py`, `facts["reconciliation_proxy"]`). It also computes monetary totals (item prices 13,591,643.70 BRL; freight 2,251,909.54 BRL; payments 16,008,872.12 BRL).

Interactions. Receives tasks from Agent 1; receives transaction amounts from Agent 2; forwards discrepancies to Agent 5; submits decisions to Agent 6; logs to Agent 7.

Governance and validation. Any adjustment or posting is a high-impact action requiring evidence and, per policy, human approval; the agent must never present an analytical total as a ledger entry.

Error handling and fallback. If orders lack either items or payments, it reports the relationship gap (Olist: 775 orders without items, 1 without payment) rather than forcing a reconciliation.

Implementation status. Proposed as an agent. The *reconciliation proxy* is implemented as a data computation; there is no accounting agent, no journal, and no ledger.

Limitations and future improvements. No general ledger, chart of accounts, tax records, settlement statements, or audited statements exist in either dataset; the Olist reconciliation is an item-versus-payment proxy only.

5.5 Agent 5 — Risk and Anomaly Detection Agent

Purpose and responsibilities. Identify unusual transactions, inconsistencies and suspicious patterns; examine cancellations, negative quantities/values, unusual payment patterns and data-quality issues; produce structured anomaly signals with explanations and confidence information where supported; prioritise cases for review; share findings with the Accounting and Governance agents.

Inputs (schema). Olist delivery-timing fields, monetary fields and review scores; Online Retail II `Quantity`, `Price`, invoice-prefix (cancellation) and missing-identifier fields.

Outputs (schema). `AnomalySignal`: entity reference, rule/method, score, explanation, evidence reference, and an explicit category (data-quality error / operational anomaly / statistical outlier / potential financial risk). **Not** "confirmed fraud".

Internal workflow. Apply transparent rules (e.g. delivered-before-carrier; negative quantity on non-cancellation invoices; zero/negative price; extreme quantity) and statistical fences (IQR) to flag candidates; attach explanations; rank by severity.

Tools, APIs, libraries, data sources. The implemented package computes the relevant inputs: numeric statistics with IQR fences and sign counts (`analysis/profiling.py`), Online Retail II quality metrics (negative/zero/negative-price counts, cancellation rows) (`analysis/online_retail.py`), and a temporal-integrity check that found 23 Olist orders recorded as delivered to the customer before carrier handoff (`analysis/olist.py`).

Interactions. Receives tasks from Agent 1; receives discrepancies from Agent 4; escalates to Agent 6; logs to Agent 7.

Governance and validation. Must label outputs as candidate anomalies; must not assert fraud; escalation is mandatory when evidence is insufficient.

Error handling and fallback. If the required distribution is unavailable, it falls back to rule-based flags and states reduced confidence.

Implementation status. Proposed as an agent. Rule inputs are implemented; no anomaly engine, scoring, or prioritisation exists.

Limitations and future improvements. No verified fraud labels exist, so precision/recall cannot be computed from these data; the agent produces candidate signals only. Terminology follows the standard anomaly-detection survey [2] and fraud-detection literature [7].

5.6 Agent 6 — Governance and Policy Enforcement Agent

Purpose and responsibilities. Validate every agent request against explicit policies; check authorization, permitted actions, required evidence and approval conditions; enforce human approval for high-impact actions; prevent unauthorized or unsupported actions; record policy decisions and reasons.

Inputs (schema). `ActionRequest`: acting agent, action type, target, justification, and evidence references. Plus the active `PolicySet` (versioned).

Outputs (schema). `PolicyDecision`: `allow` / `block` / `escalate`, the evaluated policy identifier, a reason, and (if required) a human-approval request.

Internal workflow. Evaluate the action against the applicable policies; verify evidence sufficiency; determine impact level; return a decision; if high-impact and policy says so, raise a human-approval request.

Tools, APIs, libraries, data sources. No policy engine is implemented. The dataset contains no policies, permissions or approval workflows; the policy set would be external configuration created by the prototype.

Interactions. Called by every other agent before any consequential action; interacts with Agent 7 to record decisions and with the human approver.

Governance and validation. This *is* the governance control; it must be invoked on every action and must not be bypassable by operational agents.

Error handling and fallback. Fail-closed: missing or contradictory evidence triggers escalation, never silent execution.

Implementation status. **Proposed** (design only). Candidate policy rules are listed in Section 7.

Limitations and future improvements. Because governance is conceptual, this report labels all controls as proposed; implementing and testing them is the central future milestone.

5.7 Agent 7 — Audit and Traceability Agent

Purpose and responsibilities. Record workflow events, agent decisions, tool calls, validation outcomes and policy checks; maintain links between requests, intermediate results, anomalies and final recommendations; support investigation and reproducibility; capture timestamps, identifiers, reasons and evidence references.

Inputs (schema). `AuditEvent`s emitted by all agents (see Section 10).

Outputs (schema). An append-only event stream and reconstruction queries ("what evidence justified this recommendation?").

Internal workflow. Append each event with a unique identifier, timestamp and evidence references; never mutate or delete; support deterministic replay.

Tools, APIs, libraries, data sources. No audit store is implemented. There is no append-only ledger, no cryptographic integrity check, and no tamper-proof storage. The analysis package does record input file hashes via `config.sha256_file` and writes run artifacts, which is a limited precedent for reproducibility but not an audit ledger.

Interactions. Receives events from all agents; provides traces to the Orchestrator and to a human investigator.

Governance and validation. Must record every policy decision and its reason; must not store personal data (see privacy note in Section 12).

Error handling and fallback. Detects missing audit fields and flags incomplete traces; if the store is unavailable, the workflow must fail closed rather than proceed unlogged.

Implementation status. **Proposed.** Not implemented; the report does *not* describe the audit system as immutable or tamper-proof.

Limitations and future improvements. Audit records are prototype-generated and are not attributes of either dataset.

5.8 Agent comparison

Table 1 summarises the seven agents. The "Base data" column names the dataset evidence each agent would use; the "Status" column is fixed by repository inspection.

Table 1: Summary of the seven proposed agents. Status is determined from repository evidence; all agents are proposed.

Agent	Base data	Key output	Principal limitation
1 Orchestrator	Workflow state; adapter access	Consolidated `WorkflowResult`	Coordination cannot be evidenced from historical data
2 E-Commerce	Olist orders/items/ payments/reviews; Retail lines	`OperationsSummary`	No live data; Retail lacks order status
3 Inventory	Olist items/products; Retail lines	`DemandIndicator` (proxy)	No stock levels or replenishment data
4 Accounting	Olist payments/items; Retail line value	`ReconciliationResult`	No general ledger or tax data
5 Risk / Anomaly	Timing/amount/ quantity fields	`AnomalySignal` (candidate)	No verified fraud labels
6 Governance	Policy set (external)	`PolicyDecision`	Policies are not dataset content
7 Audit	All agent events	Append-only `AuditEvent` stream	Not implemented; not immutable yet

6 Inter-Agent Communication and Coordination

Because no messaging framework exists in the repository, **no specific protocol or framework is claimed**. The following is a proposed communication design.

Task delegation. The Orchestrator emits `TaskRequest` messages naming the target agent, the task type, the data scope and the deadline. Specialists never invoke one another directly except through the Orchestrator, which prevents circular delegation.

Message formats. Structured, schema-validated messages (JSON) carrying a correlation identifier that links every message to the originating request. Candidate schemas are given in Section 10.

Shared state and workflow context. A workflow context object holds the request, the task graph and the accumulated results; it is owned by the Orchestrator and is the single writer.

Dependencies. Declared in the task graph. Independent tasks may run concurrently; dependent tasks wait.

Error propagation and retries. Failures return structured errors with retryability; transient failures may be retried with a bounded count; permanent failures degrade the workflow and are recorded.

Conflicting recommendations. When two specialists conflict (e.g. E-Commerce says "fulfil" while Risk raises a flag), the Orchestrator does not silently resolve; it escalates to Governance, which may require human adjudication.

Conditions requiring human intervention. Any action that is high-impact by policy, any escalation, and any unresolved conflict.

7 Governance and Security Model

7.1 Governance lifecycle

Figure 3 shows the proposed lifecycle applied to every agent action: request → task planning → authorization and policy validation → agent execution → result validation → risk assessment → human approval if required → audit recording → final response. A blocked or escalated action is returned to the Orchestrator with a recorded reason, and no unsafe action is executed.

Figure 3 - Proposed governance lifecycle for every agent action

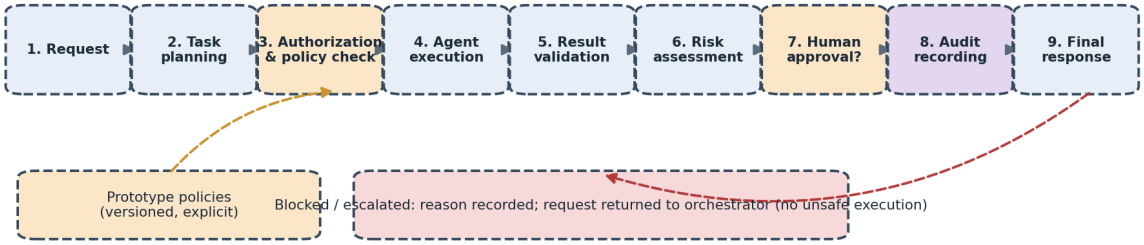


Figure 3: Proposed governance lifecycle. Every step after planning is gated; execution cannot occur without an allow decision.

7.2 Candidate policy rules

The following explicit rules are proposed for the prototype's policy set. They are *design recommendations*; none is currently enforced in code.

- **Read-only analysis** may proceed when the requester is authorized.
- **Financial adjustments** require evidence and independent validation.
- **High-impact actions** require human approval when policy specifies.
- **Missing or contradictory evidence** must trigger escalation, never silent execution.
- **Agents must not bypass governance**; consequential actions are fail-closed.

7.3 Security principles

Least privilege. Each agent receives only the data and tools its role requires; adapters are read-only.

Input validation. All messages are schema-validated before dispatch (the implemented analysis package already demonstrates schema validation for source data).

Evidence requirements. Recommendations carry evidence references; a recommendation without resolvable evidence is rejected.

Controlled tool access. Tools are exposed through explicit interfaces, not arbitrary execution.

Separation of recommendation and execution. Agents propose; Governance authorizes; only an authorized executor acts. This mirrors internal-control principles [3, 10].

7.4 What is enforced versus recommended

Enforced today: nothing at the agent layer. **Recommended:** the entire governance lifecycle, the policy rules above, and the audit linkage. The repository does not demonstrate technical enforcement of any of these, and this report does not claim otherwise. The design aligns with recognised AI-governance guidance [9, 1, 6].

8 Dataset and Agent Mapping

This section maps each agent to the available dataset fields. It does *not* repeat the dataset analysis; the authoritative reference is the project's *Dataset Report* (Deliverable 2). All counts below are measured there and reproduced by the implementation. Table 2 uses the following categories: **Direct** (a source field is directly usable), **Derived** (computed from source fields), and **Absent** (the dataset cannot supply the item).

Table 2: Agent-by-agent mapping of required data, available fields, derived features, supported tasks and unavoidable gaps.

Agent	Required data	Available (source)	Derived features	Unsupported (absent)
Orchestrator	Task/state; entity links	`order_id`, `customer_id`, invoice, dates	Task graph over linked records	Agent coordination logs (native)
E-Commerce	Orders, items, payments, reviews, stock lines	Olist status/ timestamps/ payments/ reviews; Retail invoice/ quantity/price/ date/country	Monthly volume, status mix, category revenue, cancellation rate	Live sessions; Retail order status
Inventory	Products, quantities, stock	`product_id`, item prices; `StockCode`, `Quantity`	Units sold, demand variability, velocity ranking	**Stock-on-hand, replenishment, movements**
Accounting	Payments, order value, ledger	Olist `payment_value`/`price`/ `freight`; Retail line value	Order reconciliation proxy; revenue totals	**General ledger, tax, settlement, chart of accounts**
Risk / Anomaly	Amounts, timing, quantity flags	Delivery timestamps, prices, `Quantity`, `C` prefix	IQR outliers, temporal inconsistencies, negative/zero signals	**Verified fraud labels**
Governance	Policies, permissions	Structured fields usable as rule inputs	Threshold rules (e.g. freight ratio)	**Policies, permissions, approvals**

Agent	Required data	Available (source)	Derived features	Unsupported (absent)
Audit	Event log; evidence keys	Business record keys	Evidence references; input hashes	Native audit log / chain of custody

Explicit absences. The following are **absent from both datasets** and are therefore prototype responsibilities: inventory snapshots and stock movements; verified fraud labels; general-ledger and tax data; explicit governance-policy records; and native multi-agent execution logs. Related measured caveats from the *Dataset Report*: Online Retail II has 22.77% missing customer identifiers; Olist geolocation has 26.18% exact duplicate rows; neither has returns/refunds as a first-class table.

9 End-to-End Workflow Scenarios

Four scenarios illustrate coordinated behaviour. **All four are proposed workflows**; none has been executed. Each figure is a proposed sequence diagram with decision and failure points.

9.1 Scenario A — Order and payment reconciliation

An order-related request is received; relevant order and payment records are retrieved; item totals (price + freight) are compared with payment totals per order; discrepancies beyond a threshold are escalated; governance is applied; a recommendation and audit record are produced. This scenario has the strongest empirical anchor: 99.62% of the 98,665 orders with both items and payments agree within one cent (measured). Failure cases: orders lacking items or payment (775 and 1 respectively) are reported as gaps, not reconciled.

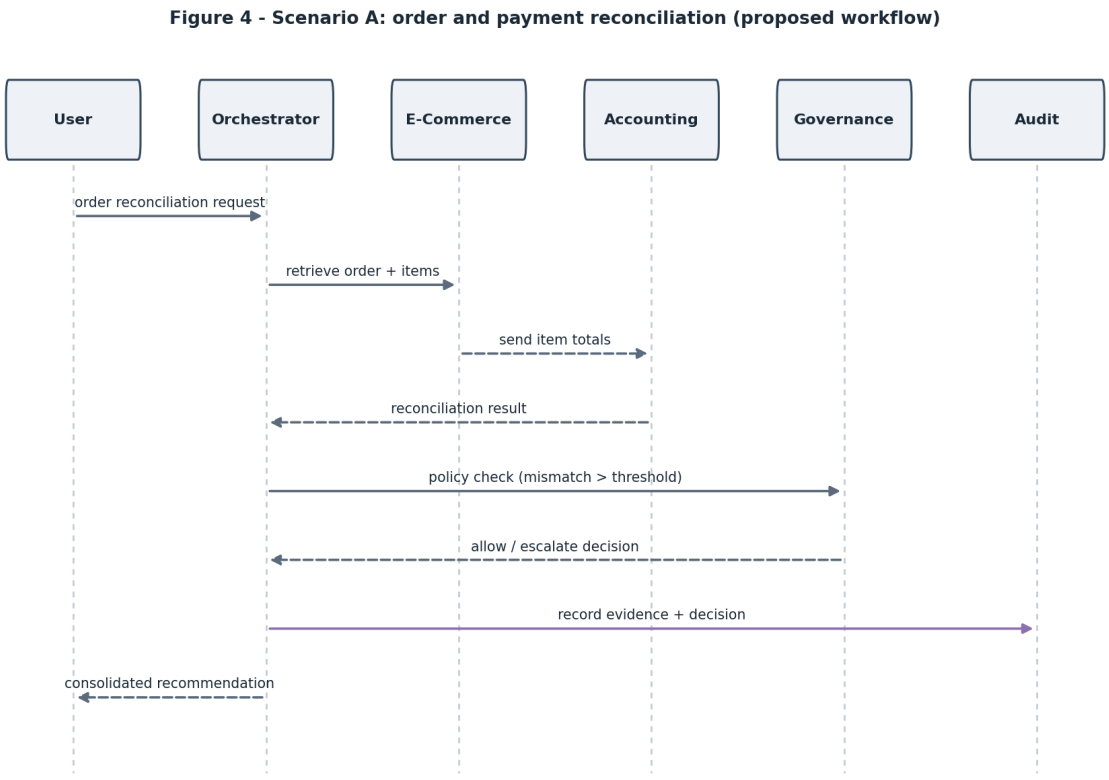


Figure 4: Scenario A (proposed): order and payment reconciliation with governance gate and audit record.

9.2 Scenario B — Transaction anomaly investigation

A transaction is analysed; a candidate anomaly is identified (e.g. a negative quantity on a non-cancellation invoice, or a delivered-before-carrier order); supporting evidence is gathered; the Accounting agent checks the relevant financial relationship; risk and governance rules are applied; the case is escalated when evidence is insufficient. This scenario is supported by the measured Online Retail II signals (12,326 and 10,624 negative-quantity lines per sheet; 8,292 cancellation invoices combined) and the 23 Olist delivered-before-carrier orders. Failure cases: the absence of fraud labels means only candidate signals can be produced.

Figure 5 - Scenario B: transaction anomaly investigation (proposed workflow)

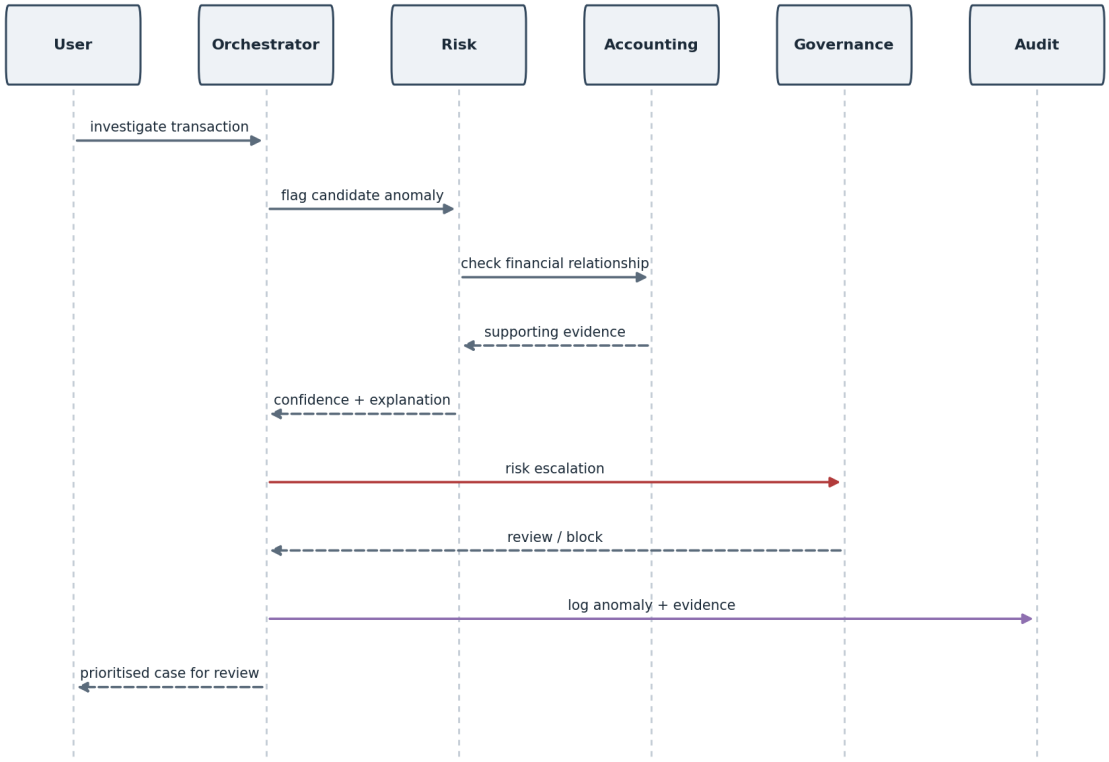


Figure 5: Scenario B (proposed): anomaly investigation with accounting cross-check and escalation.

9.3 Scenario C — Inventory risk analysis

Product and order-item patterns are analysed; demand indicators are estimated where possible; inventory-related information gaps are identified; a *qualified* recommendation is produced; and the workflow explains why actual stock decisions require additional inventory data. This scenario is deliberately limited: neither dataset contains stock-on-hand. Failure cases: missing product metadata (610 Olist products) reduces coverage.

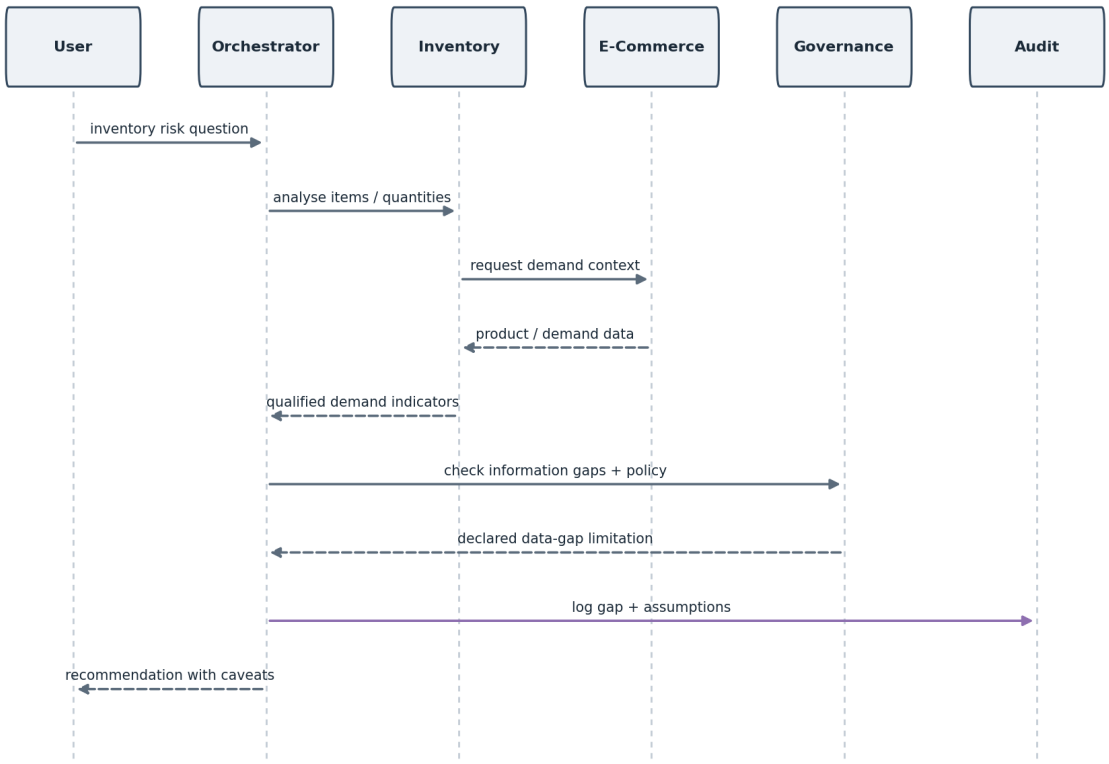


Figure 6: Scenario C (proposed): inventory risk analysis with explicit data-gap declaration.

9.4 Scenario D — Governance rejection

An agent proposes a restricted or insufficiently supported action; the Governance agent checks the policy; the action is rejected or escalated; the decision and reason are recorded for audit. This scenario exercises the fail-closed behaviour that the architecture requires. Failure cases: if the policy set is missing or contradictory, the action must be escalated, never executed.

Figure 7 - Scenario D: governance rejection (proposed workflow)

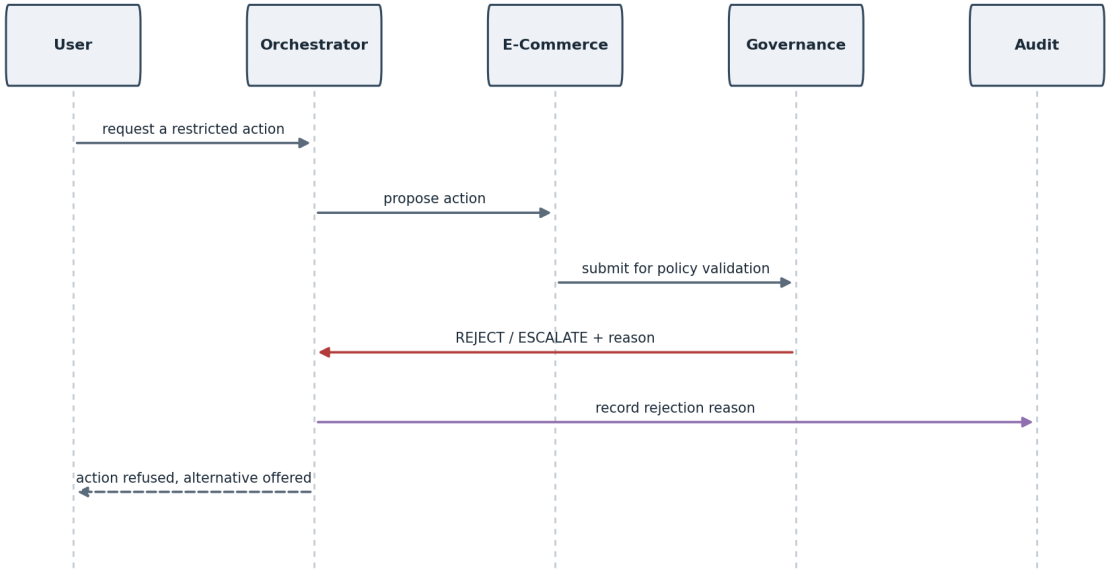


Figure 7: Scenario D (proposed): governance rejection with recorded reason and audit entry.

10 Data Contracts and Audit Schema

The repository contains **no implemented message, task, anomaly, policy or audit schemas**; what follows are *proposed* contracts. The only implemented "schemas" are the source-data column expectations in the analysis package (`olist.EXPECTED_COLUMNS`, `online_retail.EXPECTED_COLS`), which are not agent contracts. JSON examples are illustrative and clearly labelled as proposed.

Task request (proposed).

```
{ "task_id": "t-0001", "correlation_id": "req-7f3a",
  "target_agent": "accounting", "task_type": "reconcile_order",
  "scope": {"dataset": "olist", "order_id": "..."},
  "deadline": "2026-01-01T00:00:00Z" }
```

Agent result (proposed).

```
{ "task_id": "t-0001", "status": "success",
  "summary": {"matched": true, "abs_diff": 0.0},
  "evidence_refs": [{"table":"order_payments","key":"..."},
                    {"table":"order_items","key":"..."}],
  "agent": "accounting", "timestamp": "..." }
```

Anomaly finding (proposed).

```
{ "anomaly_id": "a-0012", "category": "statistical_outlier",
  "rule": "iqr_quantity", "score": 0.93,
  "explanation": "quantity beyond IQR fence",
  "evidence_refs": [{"table":"online_retail_II","key":"..."}],
  "label": "candidate_anomaly" }
```

Policy decision (proposed).

```
{ "decision_id": "p-0044", "action": "post_adjustment",
  "policy_id": "FIN-002", "decision": "escalate",
  "reason": "requires human approval",
  "human_approval_required": true }
```

Audit event (proposed). Table 3 lists the proposed audit-event fields. These events are prototype-generated; they are *not* attributes of either dataset. The schema deliberately excludes personal data.

Table 3: Proposed audit event schema (prototype-generated; not implemented).

Field	Type	Meaning
`event_id`	string (UUID)	Unique identifier of the audit event
`timestamp`	datetime	When the action occurred
`agent`	string	Agent that acted
`action`	string	Operation performed
`policy_id`	string	Experimental governance rule evaluated
`policy_decision`	enum	allow / block / escalate
`evidence_refs`	list	Source records (table + key) used as evidence
`input_digest`	string (hash)	Hash of the input records (precedent: `config.sha256_file`)
`outcome`	enum	success / failure / pending
`notes`	string	Free-text rationale (no personal data)

11 Evaluation Methodology

No agent experiments have been run, so this section is a *proposed* evaluation plan; no results are reported. The plan follows a design-science structure of build-and-evaluate [5].

Metrics. Table 4 classifies metrics by whether they can be computed immediately from data, or require the prototype, annotations, or external ground truth.

Table 4: Proposed evaluation metrics and their data requirements.

Metric	Status	Basis / requirement
Data validation error rate	Immediate	Schema/key/integrity checks (implemented)
Reconciliation mismatch rate	Immediate (proxy)	Olist item-vs-payment (99.62% within one cent measured)
Record duplication rate	Immediate	Exact-duplicate counts (implemented)
Anomaly flag rate	Immediate (rules)	Rule-based flags on quantities/prices/timestamps
Anomaly precision / recall	Requires labels	Needs trustworthy ground-truth annotations (absent)
Policy violation detection rate	Requires prototype	Depends on experimental policy definitions
False-block rate	Requires prototype	Requires prototype outcomes
Task completion rate	Requires prototype	Workflows executed by the Orchestrator
Agent coordination failures	Requires prototype	From orchestration traces
Audit event completeness	Requires prototype	Coverage over generated audit logs
Evidence traceability	Requires prototype	Fraction of decisions with resolvable evidence
Latency and computational cost	Requires prototype	Time/cost per agent run

Experimental procedure. For each scenario, run the workflow under (i) a no-agent scripting baseline, (ii) a single-agent baseline, and (iii) the full governed multi-agent configuration; compare on the metrics above. Baselines isolate the contribution of coordination and governance from the underlying analytics. Fix random seeds for any stochastic method and version-pin dependencies.

Reproducibility. Immutable raw inputs with recorded SHA-256 hashes (already produced by the analysis package), machine-readable outputs for every reported number, and a committed script that regenerates results.

Limitations. Precision/recall for anomalies is not computable without labels; governance metrics depend on policies the prototype itself defines; and single-source, single-period data limit generalisability.

12 Failure Modes and Limitations

- **Incorrect or incomplete source records.** e.g. Olist orders without items/payment/review; Retail lines without a customer identifier.
- **Missing fields and inconsistent identifiers.** Structural missingness (undelivered timestamps; 22.77% missing customer IDs), and mixed-type codes in Online Retail II.
- **Unsupported inferences.** The design forbids inventing stock levels, ledger balances or fraud labels; any such inference is a failure.
- **False positives in anomaly detection.** Candidate signals include legitimate returns/adjustments; without labels these cannot be separated by measurement alone.
- **Agent disagreement.** Resolved by escalation, never by silent override.
- **Invalid or malformed messages.** Prevented by schema validation; rejected messages fail closed.
- **Tool failures and timeouts.** Handled by bounded retries and graceful degradation.
- **Governance bypass risks.** Mitigated by making Governance a mandatory chokepoint; until implemented, this risk is open.

- **Incomplete audit trails.** Would invalidate traceability claims; the workflow must fail closed if auditing is unavailable.
- **Dataset bias and generalisability.** Both datasets cover specific retailers, regions and periods.
- **Offline-versus-live gap.** Historical replay cannot evidence autonomous live coordination.

Ethical considerations and privacy. Both datasets are anonymised/pseudonymised public releases but describe real commercial behaviour. The design therefore never reproduces personal identifiers, treats free-text reviews as potentially sensitive, and requires the audit schema to exclude personal data. Any deployment must respect data-protection obligations such as the GDPR [4] and the datasets' licences (Online Retail II: CC BY 4.0; Olist: CC BY-NC-SA 4.0).

13 Implementation Roadmap

The roadmap is ordered by dependency and adapted to actual progress. Only milestone 1 is substantially complete.

1. **Data interfaces and validation.** *Partially implemented.* The analysis package provides read-only loading, schema validation, key/integrity checks and profiling; a prototype of the ingestion/validation layer exists. Remaining: expose these as stable agent-facing interfaces and canonical events.
2. **Agent contracts and coordination.** *Proposed.* Define message/task schemas and an Orchestrator that executes a task graph.
3. **Specialised analytical capabilities.** *Proposed.* Wrap the existing computations into the E-Commerce, Inventory, Accounting and Risk agents.
4. **Governance enforcement.** *Proposed.* Implement the policy set and the allow/block/escalate chokepoint.
5. **Audit and traceability.** *Proposed.* Implement the append-only audit store and evidence linkage.
6. **Integration and end-to-end testing.** *Proposed.* Execute the four scenarios across both adapters.
7. **Evaluation and reproducibility.** *Proposed.* Run the evaluation protocol with baselines.
8. **Documentation and deployment readiness.** *Proposed.* Package, document and harden.

14 Conclusion and Future Work

This report specifies a governance-first, seven-agent architecture for autonomous e-commerce and accounting operations and documents its status honestly. The principal finding is a clear gap between design and implementation: **no agent is implemented**. The project currently contains a solid, tested data-analysis and validation layer and a dataset inspector, on which the agents can be built, but it contains no orchestration, messaging, policy, or audit code. The contribution of this report is therefore a concrete, evidence-grounded specification and roadmap, not a description of a working multi-agent system.

Future work is the roadmap itself: promote the analysis package into stable agent-facing interfaces, implement the Orchestrator and specialist agents, implement and test the governance chokepoint and the audit ledger, execute the four scenarios, and evaluate against the proposed baselines. Only after those steps could the project legitimately claim a functioning governed multi-agent system, and only with a labelled dataset could it claim anomaly-detection accuracy.

References

[1] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024.

[2] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

[3] Committee of Sponsoring Organizations of the Treadway Commission (COSO). Internal control — integrated framework. Framework, 2013.

[4] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (general data protection regulation). Official Journal of the European Union, L 119, 2016.

[5] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.

[6] International Organization for Standardization and International Electrotechnical Commission. ISO/IEC 42001:2023 information technology — artificial intelligence — management system. International Standard, Edition 1, 2023. Published 2023-12; prepared by ISO/IEC JTC 1/SC 42.

[7] Eric W. T. Ngai, Yong Hu, Y. H. Wong, Yijun Chen, and Xin Sun. The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569, 2011.

[8] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, New York, USA, 2009.

[9] Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.

[10] United States Congress. Sarbanes–oxley act of 2002. Public Law 107–204, 116 Stat. 745, 2002.

[11] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, Chichester, UK, 2 edition, 2009.

A Repository Inventory and Reproduction

A.1 What the repository actually contains

- `README.md` — empty.
- `src/` — empty (no source code).
- `datasets/olist/` — nine Olist CSV files (read-only).
- `datasets/online_retail_ii/online_retail_II.xlsx` — two worksheets (read-only).
- `reports/dataset_report/` — the *Dataset Report*: `dataset_report.tex/.pdf`, `references.bib`, and the implemented analysis package `analysis/` with `analysis_results/`, `figures/` and `tests/`.
- `reports/dataset_selection_table/` — the project/dataset overview: `main.tex/.pdf`, `scripts/inspect_datasets.py`, and CSV/JSON artifacts.

A.2 Implemented analysis package (evidence)

`reports/dataset_report/analysis/` exposes: `config` (paths, `sha256_file`, writers); `profiling` (`column_profile`, `exact_duplicates`, `numeric_stats`, `categorical_stats`, `temporal_profile`, `key_check`, `value_counts_table`); `olist` (`load_tables`, `validate_schema`, `referential_integrity`, `_join_expansion`, `run`); `online_retail` (`load_sheets`, `_clean`, `_clean_code`, `_strip_nullable`, `_invoice_kind`, `_sheet_quality`, `run`); `figures`; `compare` (`dataset_inventory`, `dataset_comparison`, `agent_suitability`). Entry point: `python -m analysis`.

A.3 Toolchain observed

Python 3.10.11; pandas 2.3.3; NumPy 1.26.4; matplotlib 3.10.7; seaborn 0.13.2; openpyxl 3.1.5; pytest 9.1.0. The dataset-selection inspector uses only the Python standard library. LaTeX (MiKTeX) with `pdflatex`/`bibtex` compiles this report.

A.4 Tests executed

The dataset-analysis tests (`reports/dataset_report/tests/test_analysis.py`) were run with `python -m pytest tests -q`; **13 tests passed**. These tests validate data-metric definitions (missing-value preservation, duplicate counting, key checks, referential integrity, sign counts, temporal profiling, cancellation detection, revenue definitions, schema validation). **No agent-level tests exist.**

A.5 Reproducing this report's diagrams

From `reports/Agents reports/figures`: run `python make_figures.py` to regenerate all diagrams as PNGs. The diagrams are schematic and are not measured data.